

Latest Zoho Array Problem Types

1. Alternate Sorting (On-Campus Round)

Description:

Rearrange an array so that the first element is the maximum, the second is the minimum, the third is the second maximum, and so on.

Input:

7

1 2 3 4 5 6 7

Output:

7 1 6 2 5 3 4

Constraints:

- Use only arrays and loops.
 - Do not use built-in sort methods.
-

2. Count Inversions (Advanced Coding)

Description:

Count the number of inversion pairs in an array, where an inversion is (i, j) such that $i < j$ and $arr[i] > arr[j]$.

Input:

5
2 4 1 3 5

Output:

3

(Explanation: Inversions are (2,1), (4,1), (4,3))

Constraints:

- No built-ins.
 - Brute force $O(n^2)$ acceptable for Level-2.
-

3. Common Elements in Three Sorted Arrays**Description:**

Given three sorted arrays, find the elements common to all three.

Input:

6
1 5 10 20 40 80
5
6 7 20 80 100
8
3 4 15 20 30 70 80 120

Output:

20 80

Constraints:

- Use three-pointer traversal.
 - Avoid extra data structures unless needed.
-

4. Sliding Window Maximum**Description:**

Given an array and window size k, print the maximum for each sliding window.

Input:

9

3 2 7 6 5 1 2 3 4

3

Output:

7 7 7 6 5 3 4

Constraints:

- No deque/collections.
 - Solve with loops or optimized logic.
-

5. Merge Two Sorted Arrays Without Duplicates

Description:

Merge two sorted arrays into one sorted array without duplicates.

Input:

```
5
1 2 3 6 9
4
2 4 5 10
```

Output:

```
1 2 3 4 5 6 9 10
```

Constraints:

- Use pointers and loops.
 - Avoid sets or built-in merging.
-

6. Majority Element (More Than $n/2$ Occurrences)

Description:

Find the element that appears more than half the time in the array, or print "None" if no such element exists.

Input:

```
9
```

1 2 3 4 5 5 5 5 5

Output:

5

Constraints:

- Boyer–Moore algorithm preferred ($O(n)$, $O(1)$ space).
 - Brute force counting acceptable.
-

Recently Asked Zoho Array-Based Problems

7. Combine & Zero-out Repeated Numbers

Description:

When a number is repeated consecutively, sum them and set the next index to 0. Push all zeros to the end.

Input:

9
0 2 2 2 0 6 6 0 8

Output:

4 2 12 8 0 0 0 0 0

Constraints:

- Only arrays and loops.
 - No collections.
-

8. Sort Odd Desc, Even Asc (Preserve Positions)**Description:**

Sort odd numbers in descending order and even numbers in ascending order, keeping odd/even positions fixed.

Input:

5
7 4 3 5 2

Output:

7 2 5 4 3

Constraints:

- Maintain position type (odd/even).
 - No built-in sort.
-

9. Next Greater Element for Each (Min)

Description:

For each element, find the smallest number greater than it in the array. If none exists, print -1.

Input:

7

2 3 7 -1 8 5 11

Output:

2->3 3->5 7->8 -1->2 8->11 5->7 11->-1

Constraints:

- Arrays and loops only.
 - No built-in search functions.
-

10. Max Sum Contiguous Subarray (No Negatives)

Description:

Find the subarray with maximum sum containing no negative numbers.

Input:

8

-2 7 5 -1 3 2 9 -7

Output:

14
3 2 9

Constraints:

- Avoid negative numbers in sum.
 - $O(n)$ scan possible.
-

11. Reverse Characters Without Moving Punctuation/Spaces**Description:**

Reverse only letters, keeping punctuation and spaces in place.

Input:

house no : 123@ cbe

Output:

ebc 32 1o : nes@ uoh

Constraints:

- No regex or built-in replaceAll.
 - Manual character checks.
-

12. Dictionary Word Search in 2D Grid (Right & Down only)

Description:

Search for words from a dictionary in a grid of letters, moving only right or down.

Input:

```
4
a z o l
n x h o
v y i v
o r s e
5
van
zoho
love
are
is
```

Output:

```
zoho
love
is
```

Constraints:

- No diagonals/backtracking.
- Only right/down moves.

13. String Word Reversal Recursively

Description:

Reverse the order of words in a string using recursion.

Input:

one two three

Output:

three two one

Constraints:

- No split/reverse built-ins.
- Must use recursion.

14. Maximum Sum Subarray (Kadane's Algorithm)**Description:**

Find the contiguous subarray with the maximum sum.

- **Input:**
`arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`
- **Output:**
`6` (subarray `[4, -1, 2, 1]`)
- **Constraints:**
 - $O(n)$ time, $O(1)$ space.

15. Sort an Array of 0s, 1s, and 2s (Dutch National Flag Problem)

Description:

Sort the array containing only 0, 1, and 2 without using built-in sort.

- **Input:**

```
arr = [0, 1, 2, 0, 1, 2]
```

- **Output:**

```
[0, 0, 1, 1, 2, 2]
```

- **Constraints:**

- $O(n)$ time, $O(1)$ space.
-

16. Longest Consecutive Sequence

Description:

Find the length of the longest sequence of consecutive numbers.

- **Input:**

```
arr = [100, 4, 200, 1, 3, 2]
```

- **Output:**

```
4 (sequence [1, 2, 3, 4])
```

- **Constraints:**

- Expected $O(n)$ time using hashing.
-

17. Trapping Rain Water

Description:

Given elevation map, find the water that can be trapped after rain.

- **Input:**

`arr = [0,1,0,2,1,0,1,3,2,1,2,1]`

- **Output:**

`6`

- **Constraints:**

- $O(n)$ time, $O(1)$ or $O(n)$ space allowed.
-

18. Subarray with Given Sum (Positive Numbers)

Description:

Find the first subarray with a given sum.

- **Input:**

`arr = [1, 4, 20, 3, 10, 5], sum = 33`

- **Output:**

`2 4` (1-based index)

- **Constraints:**

- Sliding window approach, $O(n)$ time.
-

19. Majority Element ($>n/2$ times)

Description:

Find the element that appears more than $n/2$ times (if exists).

- **Input:**

`arr = [3, 3, 4, 2, 4, 4, 2, 4, 4]`

- **Output:**

`4`

- **Constraints:**

- $O(n)$ time, $O(1)$ space (Boyer–Moore Voting Algorithm).
-

20. Pair with Given Sum

Description:

Check if there exists a pair whose sum equals the target.

- **Input:**

`arr = [8, 7, 2, 5, 3, 1], target = 10`

- **Output:**

`(8, 2) or (7, 3)`

- **Constraints:**

- $O(n)$ using hashing.
-

21. Find the Duplicate Number (Floyd's Tortoise & Hare)

Description:

Given $n+1$ integers where each number is between 1 and n , find the duplicate number.

- **Input:**

`arr = [1, 3, 4, 2, 2]`

- **Output:**

`2`

- **Constraints:**

- $O(n)$ time, $O(1)$ space.
-

22. First Missing Positive

Description:

Find the smallest positive integer missing from the array.

- **Input:**

`arr = [3, 4, -1, 1]`

- **Output:**

`2`

- **Constraints:**

- $O(n)$ time, $O(1)$ space.